

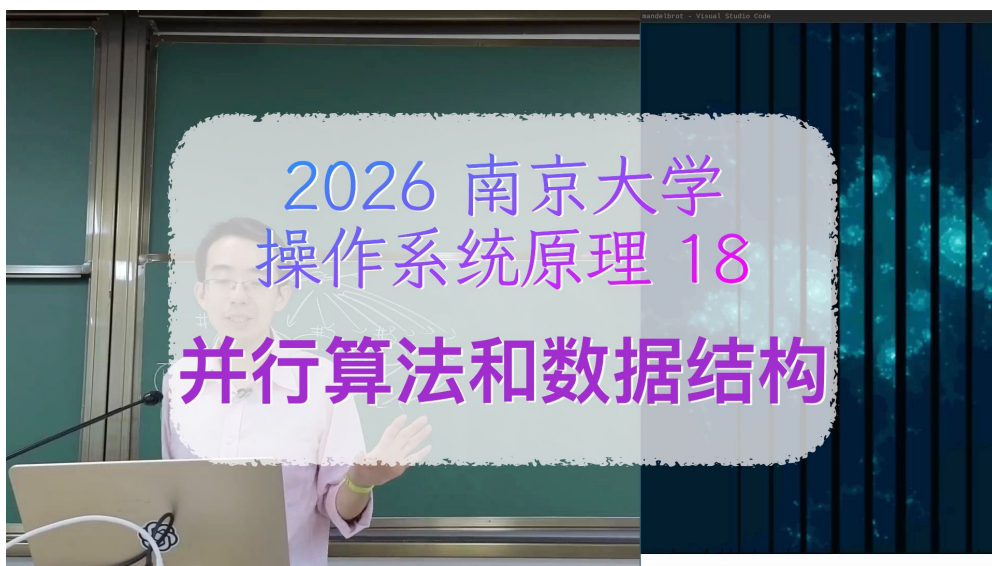
课程笔记

并行算法和数据结构

2026 南京大学《操作系统原理》第 18 讲

lecture-to-notes

2026 年 5 月 18 日



视频作者/频道：绿导师原谅你了（蒋炎岩 / jyy）

发布日期：2026-05-07

视频时长：1:26:43

视频链接：<https://www.bilibili.com/video/BV1WqdTBiEkN/>

目录

1 本讲定位：从 1 个 CPU 到 100 万个 CPU	2
1.1 开场：jyy 关于 AI 时代考试方式的反思	2
1.2 回到 sum++：为什么并发数据结构这么难	3
1.3 本章小结	3
2 并行算法：从计算图到 scale up / scale out	4
2.1 把问题画成计算图	4
2.1.1 反例：粒度太细的动态规划	4
2.1.2 正解：按对角线分块	4
2.2 High-Performance Computing：一个完整领域	5
2.2.1 物理世界天然支持并行	5
2.3 超级计算机：从 Cray-1 到 chatjimmy	6
2.3.1 今天的超算：Sunway TaihuLight & HPC-China Top 100	6
2.4 本章小结	7
3 并行数据结构：当 sum++ 真的是瓶颈	8
3.1 放松可见性：Sloppy counter	8
3.2 语法糖：thread_local	9
3.3 其他并行数据结构思想（蜻蜓点水）	10
3.4 本章小结	11
4 总结与延伸	12
4.1 讲者核心观点	12
4.2 自测三问	12
4.3 与课程其他部分的 synthesis	12
4.4 拓展阅读	12

1 本讲定位：从 1 个 CPU 到 100 万个 CPU

前面三讲（13/14/15-17）告诉了我们**怎么把多线程程序写对**——互斥、信号量、条件变量这些原语让 `sum++` 不再产生 race condition。但写对只是第一步，本讲面对的是更难的问题：**怎么把它写“快”**，并且能跨上百万核 / 上千台机器线性加速。

本讲的两个关键词

Scale up：单台机器加 CPU/核，性能跟着涨。**Scale out**：多加机器组成集群，性能跟着涨。这两个能力支撑着今天所有大规模信息系统——抖音、淘宝、大模型 serving、天气预报、AlphaFold——背后的核心引擎全是**可并行算法 + 可并行数据结构**。

1.1 开场：jyy 关于 AI 时代考试方式的反思

讲者在开头用了大约 13 分钟讨论本次期中考试的经验。要点：

- 49 份有名字的试卷，几乎一半同学交了白卷或几乎没答；
- 用 AI 直接生成讲解 slides，“一个字也没改”——**AI 对每一题都给出了高质量的回答**，甚至比讲者自己讲得更系统；
- 这引出一个开放问题：**在 Agentic AI 时代，操作系统课到底应该考什么、教什么？**

讲者的迷茫：一个值得每位学习者思考的问题

讲者反复强调：“如果我上课不教这些（fd 复制、GDB 调试细节、ELF 内部结构），你们对 AI 的失控感会越来越强；但 AI 现在每个 API 设计的 *why* 都回答得比我好——**所以学操作系统的‘品味’到底从哪里来？**”

他给出的暂时结论：你只有了解了概念，才知道用什么样的 *prompt* 让 AI 往正确方向努力——这是本讲所有“算法 / 数据结构选型”的元命题。

1.2 回到 sum++：为什么并发数据结构这么难

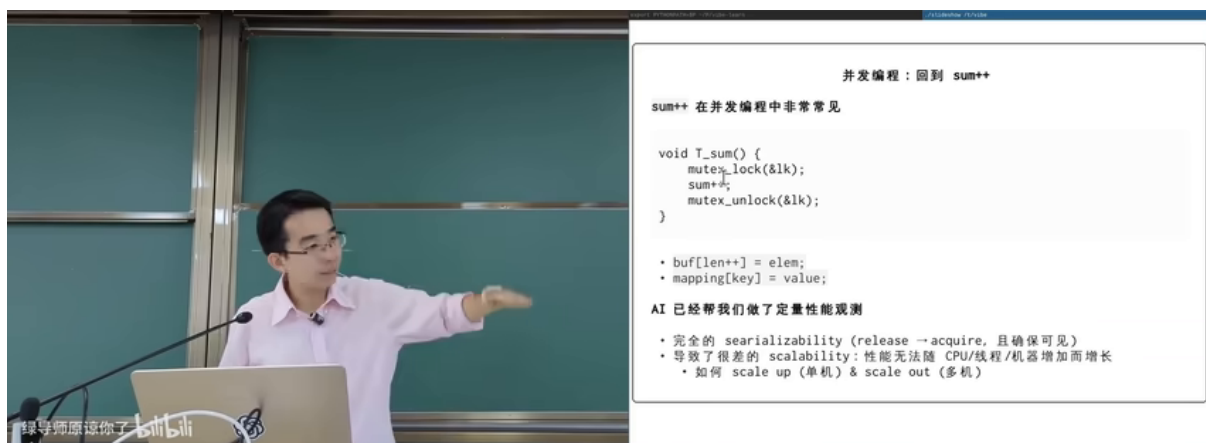


图 1: 并发编程中最常见的 pattern: sum++ 这类被互斥锁保护的极短临界区其实代表了几乎所有现实并发数据结构操作——buf[len++] = elem、mapping[key] = value 都是这个形态。AI 已经替我们做了定量观测：完整的 serializability 保证了正确性，但 scalability 极差——CPU/线程加倍，性能反而下降¹

回顾上一讲的 benchmark 结论：

- **单核**：原子指令 < mutex < spin < 每次 trap 进内核 (trap 最慢)；
- **多核 物理核数**：mutex 和 atomic 性能基本不变；
- **多核 > 物理核数**：spinlock 断崖式下跌（一核有难八核围观）；
- **所有方案都不是真正的 scale up**——理论上 N 核应该让 sum++ 快 N 倍，但实际上 mutex 路径完全不变，因为 sum++ 本质上被串行化了。

核心矛盾

互斥锁强制了一个 release→acquire 的全序关系，保证了**绝对的内存可见性和完全 serializability**——但这就意味着**彻底放弃并行**。要 scale up / scale out，我们必须**放松**这两个保证中的一部分。

1.3 本章小结

本讲不是关于”怎么写对并发”，而是关于”怎么写得既对又快”。后面会展开两条互补的思路：

1. **并行算法**——把计算图设计成”绝大部分本地、少数同步”；
2. **并行数据结构**——放松可见性保证，把全局热点数据本地化。

¹视频画面时间区间：00:20:54–00:21:09。

2 并行算法：从计算图到 scale up / scale out

2.1 把问题画成计算图

并行计算的本质模型

任何并行计算问题都可以画成一张计算图：

- 节点 = 一段计算（任意粒度——一条指令、一个矩阵乘、一个全图卷积）；
- 边 = 数据依赖关系（先算上游才能算下游）。

节点上的计算是可以独立、不需要锁的本地操作；**边上的同步**才是真正需要并发控制（互斥 / 信号量 / 条件变量）的地方。

2.1.1 反例：粒度太细的动态规划

考虑经典的 LCS / 编辑距离 / 矩阵 DP——天真实现是：

```
1 // 每一格依赖左、上、左上三个值
2 DP[i][j] = f(DP[i-1][j], DP[i][j-1], DP[i-1][j-1]);
```

Listing 1: 粒度过细的 DP（每个格子一个任务）

如果把**每一个** $DP[i][j]$ 当作一个节点，那么：

- 每个节点需要 **3 次 wait + 1 次 signal** 完成同步；
- 节点内的实际算术只有 1-3 次加法 / 比较；
- **同步代价远大于计算代价**——并行还不如串行快。

2.1.2 正解：按对角线分块

重新设计计算图——按对角线把 DP 表切成大块：

- 同一条对角线上的所有格子**无依赖**，可以完全并行；
- 跨对角线才需要同步（同步频率 = $O(\sqrt{n})$ 而非 $O(n^2)$ ）；
- 块的大小（granularity）作为可调参数——块大则同步少但并行度低，块小则反过来。最优值通常落在“同步开销 $\approx 1\%$ 计算开销”。

并行算法设计的元规则

把同步压缩到尽可能小的关键路径上，让**每一个节点上的本地计算尽可能多**。后面所有 HPC、GPU、深度学习 sharding 都是这条规则的变形。

2.2 High-Performance Computing: 一个完整领域

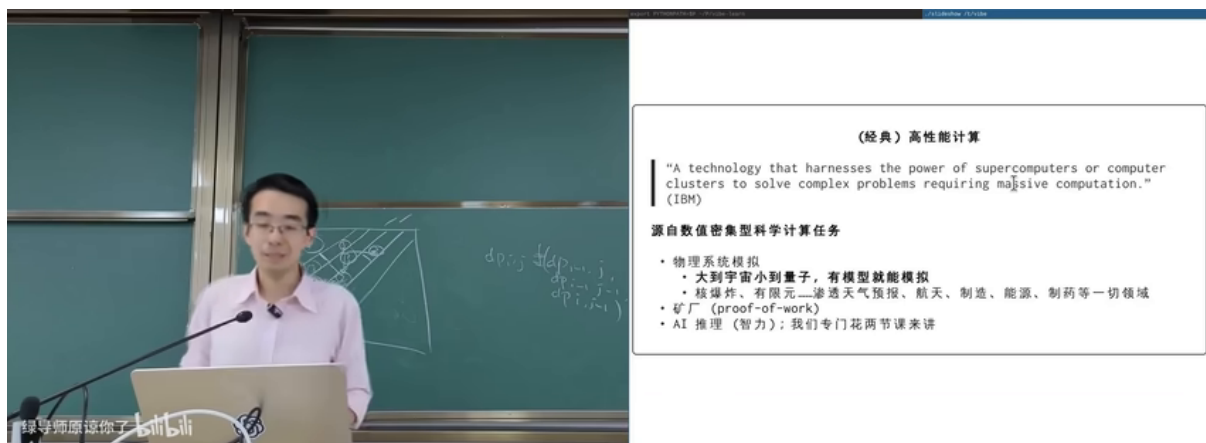


图 2: IBM 对 (经典) 高性能计算的定义: “A technology that harnesses the power of supercomputers or computer clusters to solve complex problems requiring massive computation.”——驱动力来自数值密集型科学计算: 物理系统模拟 (核爆炸、天气预报、航天、能源、新药)、矿场 (proof-of-work)、AI 推理 (占据了今天最大份额)²

2.2.1 物理世界天然支持并行

为什么 HPC 在这些问题上特别有效? 因为物理世界具有空间局部性:

- 模拟全球大气环流? 把地球切成几千万个小网格, 每个网格只和 **相邻 6 个网格** 交换边界值;
- 网格内部的计算 (牛顿力学 / Navier-Stokes 求解) 是完全本地的;
- 这天然就符合”绝大部分本地、少数边界同步”的并行模式。

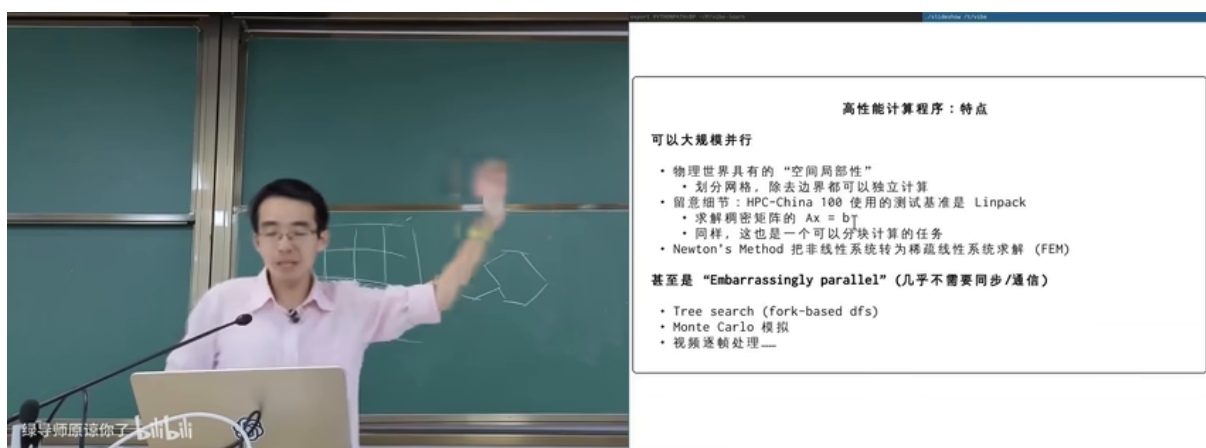


图 3: HPC 程序的两大类可大规模并行特点: (1) 利用空间局部性的 分块算法——HPC-China 100 用的基准 **Linpack** 就是求解稠密线性方程组 $Ax = b$, 可以分块计算; Newton 方法把非线性系统转为稀疏线性系统求解 (FEM)。(2) *Embarrassingly parallel* 几乎不需要同步: Tree search (fork-based DFS)、Monte Carlo 模拟、视频逐帧处理³

² 视频画面时间区间: 00:25:30–00:25:45。

³ 视频画面时间区间: 00:35:00–00:35:15。

为什么 Linpack 是 HPC 的 de facto 基准？

所有非线性物理系统都可以用 Newton 方法约化为稀疏线性方程组求解；有限元法 (FEM)、CFD、量子化学甚至 GPT 推理 (attention = matmul) 的底层都是线性代数。Linpack 测出”求解大型 $Ax = b$ 有多快” = 测出系统有多大用处。

2.3 超级计算机：从 Cray-1 到 chatjimmy

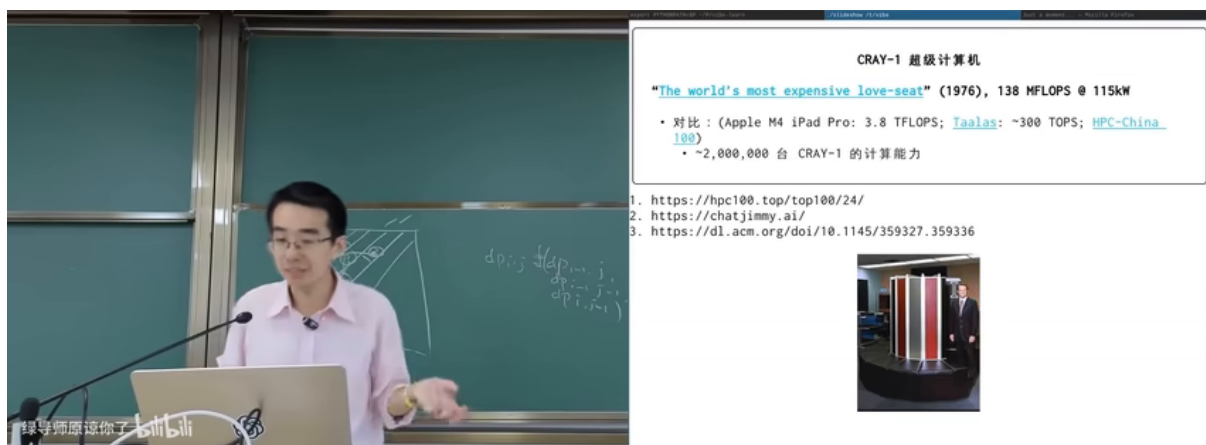


图 4: CRAY-1 (1976): "The world's most expensive love-seat"—138 MFLOPS @ 115 kW。沙发形外壳里走的是 CPU 间互联的线。对比：今天一台 Apple M4 iPad Pro ~3.8 TFLOPS (相当于 200 万台 Cray-1 的算力，能耗却只有 20 W)；ChatJimmy ASIC 把 8B 模型烧进电路，300 TOPS、~15000 tokens/s⁴

算力的 6 个数量级跃升

1976 → 2026 的 50 年里，单芯片算力提升了大约 10^6 倍：

- Cray-1 (1976): 1.38×10^8 FLOPS @ 115 kW
- Apple M4 (2024): 3.8×10^{12} FLOPS @ 20 W
- ChatJimmy ASIC (2026): 3.0×10^{14} ops @ ~30 W

推动者：科学计算（前 30 年）→ Bitcoin mining (2010s) → AI training/serving（2020s 至今，吃掉了 90%+ 算力增长）。

2.3.1 今天的超算：Sunway TaihuLight & HPC-China Top 100

- 2016 年，国产 神威·太湖之光（无锡）在 LINPACK 上夺得世界第一，93 PFLOPS；
- HPC-China Top 100 / Top500 用 PFLOPS 作为单位（比 TFLOPS 又高 3 个数量级）；
- 大量”安装地点不公开”的超算——国家战略意义；
- 但今天最贵的算力不在传统超算——而在 NVIDIA H100 / B200 集群里，全部为 LLM 训练 / serving 服务。XAI 解散，所有 GPU 流向 Anthropic 跑 Claude 系列就是最近的典型例子。

⁴视频画面时间区间：00:29:30–00:29:45。

2.4 本章小结

并行算法的核心方法论：

- **画计算图**——节点 = 本地计算、边 = 同步；
- **调粒度**——让本地计算 $\gg$ 同步开销；
- **利用局部性**——物理世界、矩阵分块、tree search 都自带局部性。

HPC 的 50 年历史就是不断地把”原本看似不能并行”的问题（求解 PDE、CFD、深度学习推理）映射到**大规模本地计算 + 少量边界同步**这个范式上。

3 并行数据结构：当 sum++ 真的是瓶颈

并行算法可以把绝大部分情形优化掉。但确实存在一类**数据结构操作非常密集**的场景——内核数据结构、数据库行锁、游戏服务器、HFT 行情、点赞计数器——此时 `sum++` 就是真正的热点，无法回避。

3.1 放松可见性：Sloppy counter

一个根本性的观察

`sum++` 用 `mutex` 实现了**严格 serializability**（“任意一次 `load` 都能立刻看到所有 `store` 的最新值”）。但很多业务场景不需要这么强：

- 微博点赞数显示晚 1 秒不要紧；
- Web 服务器的 in-flight 连接数可以是近似值；
- 进程的 CPU time 统计每秒汇总一次就够。

允许 `load` 看到“不太离谱”的旧值，但仍然保证“最终一致性”——就能换来巨大的性能提升。这就叫 **sloppy counter**。

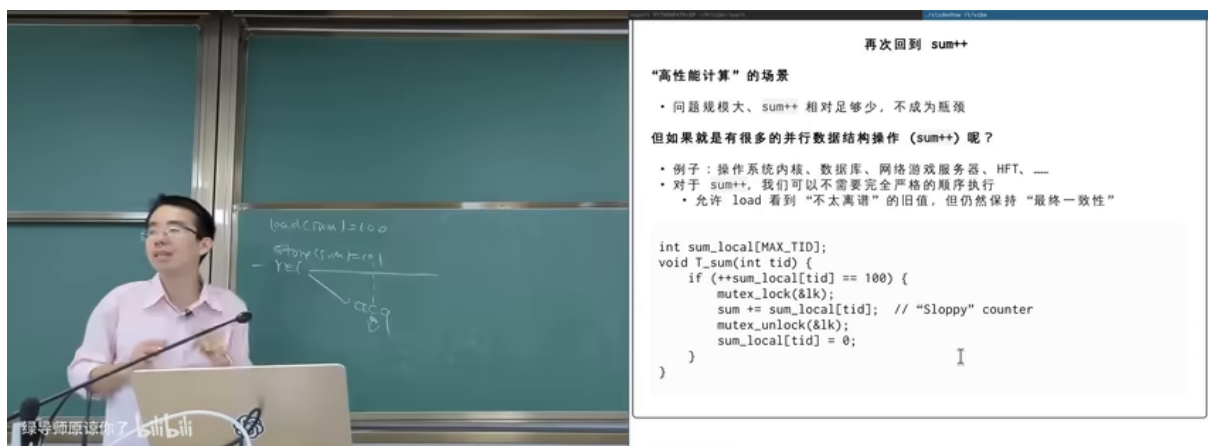


图 5: Sloppy counter 实现：每个线程一个 `sum_local[tid]`，只在 local count 达到阈值（如 100）时才上锁把它合并回全局 `sum`。99% 的更新操作**完全无锁**（线程本地内存），只有 1% 进入临界区——这是 scalability 的根本来源⁵

伪代码骨架：

```
1 #define BATCH 100
2 int sum; // 全局
3 int sum_local[MAX_TID]; // 每线程私有
4 mutex_t lk;
5
6 void T_sum(int tid) {
7     if (++sum_local[tid] == BATCH) {
```

⁵ 视频画面时间区间：01:02:30–01:02:45。

```

8     mutex_lock(&lk);
9     sum += sum_local[tid];    // 批量提交
10    mutex_unlock(&lk);
11    sum_local[tid] = 0;
12 }
13 }

```

Listing 2: Sloppy counter

两个改进方向

(1) **时间阈值**: 除了 $\text{count} \geq \text{BATCH}$, 还可以加 `timeout` (如 1 ms), 保证 reader 永远不会看到超过 1 ms 的旧值。(2) **自适应 BATCH**: 竞争激烈时 BATCH 增大 (更少同步), 空闲时减小 (更高准确度)。Linux per-cpu counter (`percpu_counter_*`) 就用了类似策略。

3.2 语法糖: `thread_local`

手写 `sum_local[MAX_TID]` 又丑又容易出错——你必须自己分配数组、管理线程号、防止伪共享 (false sharing)。

C11 / C++11 引入了关键字 `_Thread_local` (C23 起又叫 `thread_local`), 让编译器和操作系统自动为你管理**每线程一份**的变量副本:

```

1  #include <threads.h>           // C11
2  thread_local int sum_local;    // 每线程自动一份
3
4  void T_sum(int tid) {
5      if (++sum_local == BATCH) {
6          mutex_lock(&lk);
7          sum += sum_local;
8          mutex_unlock(&lk);
9          sum_local = 0;
10     }
11 }

```

Listing 3: `thread_local` 改写 sloppy counter

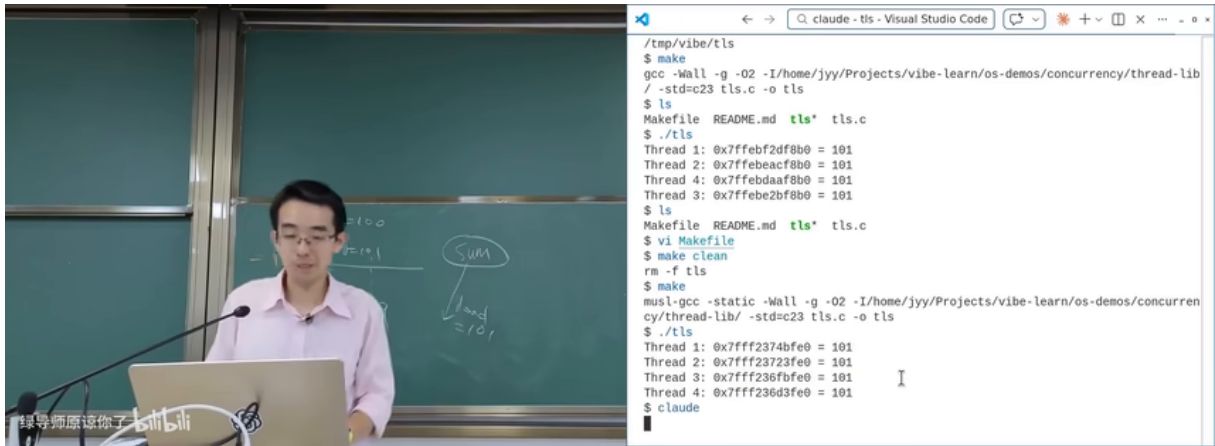


图 6: `thread_local` 真的是每线程独立地址的验证: 4 个线程同时打印自己的 `&tls`, 得到 `0x7ffebf2df8b0` / `0x7ffebecaf8b0` / `0x7ffebdaff8b0` / `0x7ffeb2bf8b0`——四个不同的地址, 但每个线程的值都是 101。可以编译到 `musl-static` binary, 证明它不需要 `glibc` 的特殊支持⁶

thread_local 的两个使用约束

(1) 必须文件作用域或 `static`——在函数作用域里写 `thread_local int x`; 编译器会直接报错 (一个 `stack frame` 里的局部变量天然就是每线程独立的, 没有意义)。(2) 实现机制是 **TLS (Thread-Local Storage)**——通过段寄存器 `%fs` (x86_64) / `TPIDR_EL0` (ARM64) 偏移寻址。访问速度等同于普通全局变量, 但调试器看到的地址会随线程不同。

3.3 其他并行数据结构思想 (蜻蜓点水)

讲者最后强调, **sloppy counter** 只是冰山一角。同一类”放松保证换性能”的思路, 在并行数据结构设计里有大量变形:

- **Lock-free linked list / queue**: 用 CAS 替代 mutex;
- **Hazard pointers / RCU (Read-Copy-Update)**: 读者完全无锁、只在写者修改时延后内存回收;
- **Per-CPU 分配器**: 每核一个 freelist, 跨核迁移走 batch;
- **Sharded counters / locks**: 把 hot key 哈希到 N 个桶。

每一个都是”把工作留在本地、把同步推到边界”的同一条规则在不同约束下的展开。

Malloc 实验里你会用到

本周的 `my_malloc` 实验需要支持多线程并发分配 / 释放。一把大锁能让你 **Easy Test** 通过, 但 **Hard Test** (高压应力 stress 测试) 要求 scalability。此时你不得不引入 `per-thread / per-CPU freelist + 中央”大池”`——就是 `sloppy counter` 的孪生兄弟。

⁶视频画面时间区间: 01:09:00–01:09:15。

3.4 本章小结

并行数据结构的核心原则：

- **放松一致性**：很多场景”最终一致”就够了；
- **本地优先**：把热数据复制到每线程 / 每核；
- **批量提交**：把高频小操作攒成低频大操作再走临界区；
- **语言原生支持**：`thread_local` 让 sloppy 模式不再”丑”。

4 总结与延伸

4.1 讲者核心观点

1. **Mutex 写对很容易，写“快”很难**——单凭 mutex 永远没有 scalability，必须放松某些保证。
2. **并行算法 = 计算图设计**——节点本地、边界同步；物理世界天然提供局部性，所以 HPC 在科学计算上特别有效。
3. **Scale up 和 scale out 是同一种思路在不同尺度上的展开**——单机内的线程并行 vs 多机集群的分布式并行，方法论一致。
4. **Sloppy 是并发数据结构性能的源泉**——sum local、per-CPU、RCU 都是同一条规则的具体化。
5. **AI 时代的“操作系统课”价值在‘品味’**——AI 能回答每一个细节，但选哪种算法、调哪个参数、做哪些放松，这些“品味”决策还得人来。

4.2 自测三问

看完本讲，能答上这三题吗？

1. 为什么用 mutex 包住的 `sum++` 即使多核执行也几乎不能加速？能用什么实验观察到这一点？
2. 用 `thread_local` 重写 sloppy counter；想一想：如果某一个线程创建很多但极少做 `++`，全局 `sum` 的实时性会有什么问题？怎么用 `timeout` 解决？
3. Linpack 为什么能成为 HPC Top 500 的事实基准？它能不能反映你的 web 服务器的真实性能？为什么？

4.3 与课程其他部分的 synthesis

- 与第 14 讲（互斥）：14 讲建立了 mutex 的概念和实现；18 讲告诉你“mutex 不是终点——它只解决了正确性，没解决 scalability”。
- 与第 15-17 讲（条件变量 / 信号量 / 并发 bugs）：这些原语让我们能表达更复杂的同步模式；18 讲告诉你怎么设计能让这些原语用得少、效率高的算法。
- 与即将到来的 19/20/21 讲：协程（轻量级线程）把 19 讲带到用户态调度——很多 web 后端的 scale up 路径用的不是线程而是协程；SIMD/GPU 是另一种“per-core 局部 + 边界同步”的硬件极致形态；Token 的旅程串起整个 AI serving 栈的数据流，每一步都印证本讲的并行原则。

4.4 拓展阅读

- Russell, *The World's Most Expensive Love Seat*, Cray Research, 1976 (Cray-1 设计图与系统介绍)。

- Hennessy & Patterson, *Computer Architecture: A Quantitative Approach*, 6th ed ——第 5 章 Thread-Level Parallelism 是本讲的硬件配套读物。
- Boyd-Wickizer *et al.*, *An Analysis of Linux Scalability to Many Cores*, OSDI 2010 ——sloppy counter 在 Linux 内核里的真实落地。
- `man 3 pthread_setspecific`, `man 7 pthreads`, `_Thread_local` (C11/C23), `percpu_counter_*` (Linux kernel).
- Top500 / HPC-China Top 100: <https://top500.org/> 与 <https://hpc100.top/top100/24/>.
- `chatjimmy.ai` ——8B 模型烧 ASIC 的 token-per-second 体验.